Atelier pratique n°3 : Les bases de Git

Niveau : Débutant · Durée : 1 h 30 · Travail individuel

Objectif de l'atelier

À la fin de cet atelier, le stagiaire sera capable de créer un dépôt Git, suivre les modifications des fichiers, enregistrer des versions et consulter l'historique du projet.

Partie 1 — Configuration et création du projet (15 min)

Travail demandé

1. Vérifier que Git est installé sur votre ordinateur.
2. Configurer votre identité Git.
3. Créer un dossier nommé `atelier-git`.
4. Initialiser un dépôt Git dans ce dossier.
5. Afficher l'état du dépôt.

Commandes à utiliser :

```
git --version
git config --global user.name "Votre Nom"
git config --global user.email "votre@email.com"

mkdir atelier-git
cd atelier-git
git init
git status
```

Questions :

- À quoi sert la commande `git init` ?
- Quel est le rôle des commandes `git config` ?
- Que signifie l'expression working directory ?

Partie 2 — Suivi des fichiers et premier commit (20 min)

Travail demandé

1. Créer un fichier `index.html` contenant le titre « Mon premier projet Git ».
2. Afficher l'état du dépôt.
3. Ajouter le fichier à la zone de staging.
4. Vérifier son état.
5. Enregistrer le premier commit avec le message `Initialisation du projet`.
6. Consulter l'historique.

Commandes à utiliser :

```
git status
git add index.html
git status
git commit -m "Initialisation du projet"
git log
```

Questions :

- Quelle est la différence entre `git add` et `git commit` ?
- Quel est le rôle de la zone de staging ?
- À quoi sert l'option `-m` ?

Partie 3 — Modifier un fichier (20 min)

Travail demandé

1. Modifier `index.html` en ajoutant un paragraphe de présentation.
2. Exécuter `git status`.
3. Afficher les modifications avec `git diff`.
4. Ajouter les modifications au staging.
5. Créer un deuxième commit avec le message `Ajout de la présentation`.
6. Afficher l'historique des commits.

Commandes à utiliser :

```
git status
git diff
git add index.html
git diff --staged
git commit -m "Ajout de la présentation"
git log --oneline
```

Questions :

- Que montre `git diff` ?
- Quelle est la différence entre `git diff` et `git diff --staged` ?
- Pourquoi créer plusieurs commits au lieu d'un seul ?

Partie 4 — Gérer plusieurs fichiers (20 min)

Travail demandé

1. Créer un fichier `style.css`.
2. Créer un fichier `README.md` décrivant brièvement le projet.
3. Afficher les fichiers suivis et non suivis.
4. Ajouter les deux fichiers au staging en une seule commande.
5. Créer un troisième commit.
6. Afficher l'historique complet sous forme compacte.

Commandes à utiliser :

```
git status
git add style.css README.md
git status
git commit -m "Ajout du style et de la documentation"
git log --oneline
```

Questions :

- Quelle est la différence entre un fichier tracked et untracked ?
- Peut-on ajouter plusieurs fichiers avec une seule commande `git add` ?
- À quoi sert le fichier `README.md` ?

Partie 5 — Bilan et vérification (15 min)

Complétez le tableau suivant.

Commande	Rôle
`git init`	À compléter
`git status`	À compléter
`git add`	À compléter
`git commit`	À compléter
`git log`	À compléter
`git diff`	À compléter

Livrable attendu :

- Un dossier `atelier-git` contenant `index.html`, `style.css` et `README.md`.
- Un dépôt Git local initialisé.
- Au moins trois commits avec des messages explicites.
- Une capture d'écran de l'historique obtenu avec `git log --oneline`.